

```
# 查看 Python 版本
import sys
print(f"Python 版本: {sys.version}")

# 查看 pandas 版本
import pandas as pd
print(f"pandas 版本: {pd.__version__}")

# 查看 NumPy 版本
import numpy as np
print(f"NumPy 版本: {np.__version__}")

# 查看其他常用库版本
import matplotlib
print(f"Matplotlib 版本: {matplotlib.__version__}")

import scipy
print(f"SciPy 版本: {scipy.__version__}")

# 如果您使用 scikit-learn
try:
    import sklearn
    print(f"scikit-learn 版本: {sklearn.__version__}")
except ImportError:
    print("scikit-learn 未安装")
```

数据准备与环境设置

```
# -*- coding: utf-8 -*-
import pandas as pd
import numpy as np
import lightgbm as lgb
from sklearn.metrics import roc_auc_score, roc_curve
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

# --- Environment Settings ---
# Set plot style
sns.set_style('whitegrid')
# Ignore warnings for cleaner output
warnings.filterwarnings('ignore')
```

```
df = pd.read_csv('./high_value_dataset.csv')
```

```
df.flag1.value_counts()  
df[df.high_salary > 0].shape  
df[df.high_salary > 0].groupby('flag1').high_salary.count()  
df.shape  
df[df.curr_salary > 0].shape  
df[df.weeks_employed_cal_mean > 0].shape  
df[df.curr_salary > 0].head()  
df.head()  
df.columns.tolist()  
df[df.curr_salary > 0].shape  
df[df.curr_salary > 1000].shape[0]/9460  
df[df.curr_salary > 0].curr_salary
```

```
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# 假设 df 已经加载并包含 curr_salary 列  
# 1. 描述性统计  
salary_distribution = df[df.curr_salary > 0]['curr_salary'].describe()  
print("当前薪资分布的描述性统计:")  
print(salary_distribution)  
  
# 可视化 - 箱线图  
plt.figure(figsize=(10, 6))  
sns.boxplot(x=df[df.curr_salary > 0]['curr_salary'])  
plt.title('当前薪资分布箱线图')  
plt.xlabel('当前薪资')  
plt.grid()  
plt.show()
```

```
df2 = pd.read_csv('./income_df6.csv')  
df = df.merge(df2[['apply_uuid', 'app_office_cnt', 'app_shebao_cnt',  
    'app_facechange_cnt']], how = 'left', on = 'apply_uuid')  
df.to_csv('high_value_dataset.csv', index = False)
```

```
df.groupby('flag1').high_salary.mean()  
df.groupby('flag1').dpd7_due45ever.mean()
```

```
df.groupby('flag1').fpd7.mean()
```

```
fea_cols = ['local_user_sim_duration_id',
'local_user_phone_duration_id',
'has_historical_figure',
'compound_name_state',
'app_investment_cnt',
'rom_total_new',
'app_ecommerce_cnt',
'ram_total_new',
'ram_new',
'network_operator_name_new',
'sim_operator_name_new',
'app_smarthome_cnt',
'local_user_salary_range_id',
'app_ewallet_cnt',
'price',
'local_user_gender_new',
'local_current_equipment_ramtotal',
'app_travel_cnt',
'screen_scaled_density',
'sim_operator',
'cpu_min_freq_new',
'local_user_max_sub_id',
'network_operator',
'battery_voltage',
'local_user_postcode',
'bajio_region',
'screen_xdpi',
'local_current_equipment_screen_ydpi',
'screen_ydpi',
'local_user_education_id',
'app_loan_cnt',
'local_device_sms_auth',
'screen_dpi',
'local_user_working_years_id',
'app_delivery_cnt',
'has_gaming',
'screen_width',
'has_social_media',
'app_carservice_cnt', 'green_geo_distance4', 'low_geo_distance1', 'low_geo_dist
ance7', 'app_office_cnt', 'app_shebao_cnt', 'app_facechange_cnt']
len(fea_cols)
```

```

# --- 1.2 Data Splitting ---
print("--- 1.2 Splitting data into train, test, and OOT sets ---")

# Define sets based on 'flag1' column
train_df = df[df['flag1'] == '0train'].copy()
test_df = df[df['flag1'] == '1test'].copy()
oot_df = df[df['flag1'] == '2oot'].copy()

# Prepare feature matrices (X) and target vectors (y)
# Primary target: high_salary
X_train = train_df[fea_cols]
y_train = train_df['high_salary']

X_test = test_df[fea_cols]
y_test = test_df['high_salary']

X_oot = oot_df[fea_cols]
y_oot = oot_df['high_salary']

# Secondary target (for later evaluation): dpd7_due45ever
y_train_dpd = train_df['dpd7_due45ever']
y_test_dpd = test_df['dpd7_due45ever']
y_oot_dpd = oot_df['dpd7_due45ever']

print(f"Train set shape: {X_train.shape}")
print(f"Test set shape: {X_test.shape}")
print(f"OOT set shape: {X_oot.shape}")

```

模型训练

在这一章，我们将使用LightGBM算法来训练模型。为了解决样本不平衡问题，我们将使用scale_pos_weight参数。为了防止过拟合，我们将采用以下策略：

使用验证集进行提前停止（Early Stopping）：在训练过程中监控测试集上的性能，当性能不再提升时停止训练。设置保守的超参数：限制树的深度和叶子节点数量，并使用子采样来增加模型的泛化能力。

```

# Calculate scale_pos_weight to handle class imbalance
# scale_pos_weight = count(negative_class) / count(positive_class)
neg_count = (y_train == 0).sum()
pos_count = (y_train == 1).sum()

```

```
scale_pos_weight_value = neg_count / pos_count if pos_count > 0 else 1
print(f"Calculated scale_pos_weight: {scale_pos_weight_value:.2f}")
```

```
# --- 2.1 Model Training ---
```

```
print("\n--- Chapter 2: Model Training ---")
```

```
# Calculate scale_pos_weight to handle class imbalance
# # scale_pos_weight = count(negative_class) / count(positive_class)
# neg_count = (y_train == 0).sum()
# pos_count = (y_train == 1).sum()
# scale_pos_weight_value = neg_count / pos_count if pos_count > 0 else 1
```

```
# print(f"Calculated scale_pos_weight: {scale_pos_weight_value:.2f}")
```

```
# Configure LightGBM model parameters
# Parameters are chosen to build a robust model and prevent overfitting
# 详细的LightGBM参数设置
```

```
lgb_params = {
    # === 核心参数 ===
    'objective': 'binary',          # 二分类任务
    'metric': 'auc',                # 评估指标为AUC
    'boosting_type': 'gbdt',        # 梯度提升决策树
    'random_state': 42,             # 随机种子，确保结果可复现

    # === 学习控制参数 ===
    'learning_rate': 0.05,          # 学习率，较小值可以提高模型稳定性
    'n_estimators': 1000,           # 最大迭代轮数
    'early_stopping_rounds': 50,    # 早停轮数，防止过拟合

    # === 树结构参数 ===
    'num_leaves': 5,                # 叶子节点数，控制模型复杂度
    'max_depth': 2,                 # 最大深度，-1表示不限制
    'min_child_samples': 20,         # 叶子节点最小样本数
    'min_child_weight': 0.001,      # 叶子节点最小权重
    'min_split_gain': 0.0,           # 分裂最小增益
    'max_bin': 5,                   # 分箱数量

    # === 特征采样参数 ===
    'feature_fraction': 0.9,         # 每次迭代使用的特征比例
    'bagging_fraction': 0.8,         # 每次迭代使用的样本比例
    'bagging_freq': 5,               # bagging频率
    'feature_fraction_bynode': 0.8,  # 每个节点使用的特征比例

    # === 正则化参数 ===
    'reg_alpha': 0.5,               # L1正则化
```

```

    'reg_lambda': 0.5,                # L2正则化

    # === 类别不平衡处理 ===
    'is_unbalance': True,             # 自动处理类别不平衡
    # 'scale_pos_weight': 5,          # 正样本权重，可根据样本比例调整

    # === 性能优化参数 ===
    'n_jobs': -1,                     # 使用所有CPU核心
    'verbose': -1,                     # 不输出训练过程信息
    'force_col_wise': True,            # 强制列式训练，提高速度
    'deterministic': True,             # 确保结果可重现
}

# Initialize the model
model = lgb.LGBMClassifier(**lgb_params)

# --- Prepare evaluation sets for monitoring during training ---
# The first set in this list is used for early stopping.
# We put 'test' first to prevent overfitting on the training data
# and to keep OOT as a final hold-out validation set.
eval_sets = [
    (X_test, y_test),
    (X_train, y_train),
    (X_oob, y_oob)
]

# Note: The sklearn wrapper for older LightGBM versions names the sets
# sequentially.
# In the output log:
# valid_0 -> will correspond to our Test set
# valid_1 -> will correspond to our Train set
# valid_2 -> will correspond to our OOT set

print("\nStarting model training with live monitoring on Train, Test, and
OOT sets.")
print("Early stopping is based on the Test set's performance (valid_0).")

# Train the model using the early_stopping_rounds parameter in fit()
# This is the common practice for older scikit-learn/lightgbm versions.
# We set verbose=50 to print evaluation results every 50 rounds.
model.fit(X_train, y_train,
          eval_set=eval_sets,
          eval_metric='auc',
          # Note: The 'callbacks' argument is the modern approach.
          # For older versions, 'early_stopping_rounds' is used directly.
          early_stopping_rounds=200,

```

```

        verbose=1)

print("\nModel training completed.")
# The best_iteration_ attribute stores the optimal number of trees found.
print(f"Best iteration found: {model.best_iteration_}")

```

模型效果评估-针对高价值标签

```

# --- 3.1 Model Performance Evaluation on Primary Target (high_salary) ---
print("\n--- Chapter 3: Model Evaluation on Primary Target (high_salary) ---")

# Helper function to calculate KS statistic
def calculate_ks(y_true, y_proba):
    """
    Calculates the Kolmogorov-Smirnov (K-S) statistic.

    Args:
        y_true (array-like): True binary labels.
        y_proba (array-like): Predicted probabilities for the positive
class.

    Returns:
        float: The K-S statistic.
    """
    # Create a dataframe with true labels and predicted probabilities
    data = pd.DataFrame({'y_true': y_true, 'y_proba': y_proba})
    data = data.sort_values('y_proba', ascending=False)

    # Calculate cumulative percentage of positive and negative samples
    # Cumulative Positive Rate (TPR)
    data['cumulative_pos'] = data['y_true'].cumsum() / data['y_true'].sum()
    # Cumulative Negative Rate (FPR)
    data['cumulative_neg'] = (1 - data['y_true']).cumsum() / (1 -
data['y_true']).sum()

    # The K-S statistic is the maximum difference between the two cumulative
distributions
    ks_value = np.max(np.abs(data['cumulative_pos'] -
data['cumulative_neg']))
    return ks_value

# --- 3.2 Calculate Metrics and Display Results ---
print("Calculating AUC and K-S statistics for all datasets...")

```

```

# Predict probabilities on all datasets
y_train_pred_proba = model.predict_proba(X_train)[: , 1]
y_test_pred_proba = model.predict_proba(X_test)[: , 1]
y_oot_pred_proba = model.predict_proba(X_oot)[: , 1]

# Calculate AUC scores
auc_train = roc_auc_score(y_train, y_train_pred_proba)
auc_test = roc_auc_score(y_test, y_test_pred_proba)
auc_oot = roc_auc_score(y_oot, y_oot_pred_proba)

# Calculate KS scores
ks_train = calculate_ks(y_train, y_train_pred_proba)
ks_test = calculate_ks(y_test, y_test_pred_proba)
ks_oot = calculate_ks(y_oot, y_oot_pred_proba)

# Display results in a structured table
print("\n" + "="*50)
print("          Model Performance Summary")
print("="*50)
print(f"| Dataset |      AUC      |      KS      |")
print(f"|-----|-----|-----|")
print(f"| Train   | {auc_train:.4f} | {ks_train:.4f} |")
print(f"| Test    | {auc_test:.4f}  | {ks_test:.4f}  |")
print(f"| OOT     | {auc_oot:.4f}   | {ks_oot:.4f}   |")
print("="*50)

# --- 3.3 Plot ROC Curve ---
# Calculate ROC curve points for visualization
fpr_train, tpr_train, _ = roc_curve(y_train, y_train_pred_proba)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_pred_proba)
fpr_oot, tpr_oot, _ = roc_curve(y_oot, y_oot_pred_proba)

plt.figure(figsize=(10, 8))
plt.plot(fpr_train, tpr_train, label=f'Train (AUC = {auc_train:.4f})')
plt.plot(fpr_test, tpr_test, label=f'Test (AUC = {auc_test:.4f})')
plt.plot(fpr_oot, tpr_oot, label=f'OOT (AUC = {auc_oot:.4f})', linestyle='--')

plt.plot([0, 1], [0, 1], color='grey', linestyle=':', label='Random Guess')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for high_salary Prediction')
plt.legend()
plt.grid(True)
plt.show()

```

模型区分能力评估 - 针对逾期标签

```
# --- 4.1 Cross-Validation on Risk Metric (is_overdue) ---
print("\n--- Chapter 4: Cross-Validation on Risk Metric (is_overdue) ---")

# Define the secondary target variable from the OOT set
y_oout_is_overdue = oot_df['dpd7_due45ever']

# --- 4.2 Calculate AUC and KS on the Risk Target ---
# Note: We are using the scores from the model trained on 'high_salary'
# to see how well it separates 'is_overdue'.

# Calculate AUC for the 'is_overdue' target
auc_overdue = roc_auc_score(y_oout_is_overdue, y_oout_pred_proba)

# Calculate KS for the 'is_overdue' target using the previously defined
function
ks_overdue = calculate_ks(y_oout_is_overdue, y_oout_pred_proba)

print("\nEvaluating the 'high_salary' model's ability to separate overdue
users on OOT set:")
print("="*60)
print(f"AUC for 'is_overdue' separation: {auc_overdue:.4f}")
print(f"K-S for 'is_overdue' separation: {ks_overdue:.4f}")
print("="*60)

# --- 4.3 Visualize Score Distribution for Overdue vs. Non-Overdue ---
# Create a temporary dataframe for easy plotting
oot_scores_df = pd.DataFrame({
    'score': y_oout_pred_proba,
    'is_overdue': y_oout_is_overdue
})

# Separate scores for the two groups
score_overdue = oot_scores_df[oot_scores_df['is_overdue'] == 1]['score']
score_non_overdue = oot_scores_df[oot_scores_df['is_overdue'] == 0]['score']

plt.figure(figsize=(12, 7))

# IMPORTANT FIX: Using 'shade=True' for compatibility with older seaborn
versions
```

```
# If you have seaborn >= 0.11, you can use 'fill=True'
print("\nPlotting score distributions... (Using shade=True for
compatibility)")
sns.kdeplot(score_non_overdue, label='Non-Overdue (dpd=0)', shade=True)
sns.kdeplot(score_overdue, label='Overdue (dpd>0)', shade=True)

plt.title('Score Distribution for Overdue vs. Non-Overdue Users (OOT Set)')
plt.xlabel('Predicted high_salary Score')
plt.ylabel('Density')
plt.legend()
plt.grid(True)
plt.show()
```

特征重要性

```
## # --- 5.1 Feature Importance Analysis ---
print("\n--- Chapter 5: Feature Importance Analysis ---")

# Create a dataframe of feature importances
feature_importance_df = pd.DataFrame({
    'feature': fea_cols,
    'importance': model.feature_importances_
}).sort_values('importance', ascending=False)

# Display the top 20 most important features
print("Top 20 most important features:")
print(feature_importance_df.head(20))

# Plot the feature importances
plt.figure(figsize=(12, 8))
sns.barplot(x='importance', y='feature',
data=feature_importance_df.head(20))
plt.title('Top 20 Feature Importances')
plt.xlabel('Importance Score')
plt.ylabel('Feature Name')
plt.grid(True)
plt.show()
```

说明：这里lgbm使用的sklearn的接口。lgbm在python中有两套接口，一是sklearn，二是原生接口。r中只有一套原生接口